

Distributed Rate Limiter: Project Plan

A phased implementation plan, architecture, and design reference. Timeline: about 6 weeks.

1. Overview

Goal: Build a horizontally scalable rate limiting service and a gateway that uses it, enforcing per-user, per-IP, and per-API-key limits correctly across multiple instances.

Target outcome: 5k+ req/s across 3 gateway instances, p99 decision overhead under 20 ms, zero limit violations under concurrent multi-instance load, graceful behavior when Redis fails.

Functional requirements

- Limit by API key, user ID, or IP; multiple rules per route; tiered limits (free/pro)
- Pluggable algorithms: fixed window, sliding window log, sliding window counter, token bucket
- Standard responses: 429, Retry-After, X-RateLimit-Limit/Remaining/Reset
- Rules configurable in YAML, later updatable at runtime

Non-functional requirements

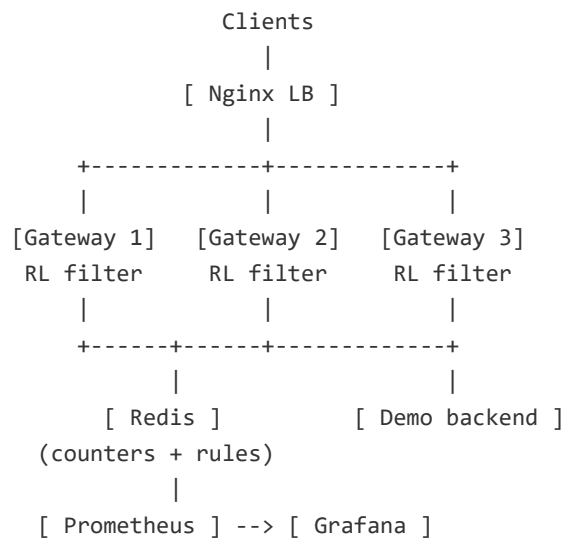
- Correctness: total allowed requests never exceed the limit across instances
- Low latency: one Redis round trip per decision
- Resilience: configurable fail-open or fail-closed on Redis outage
- Observability: metrics for allowed, blocked, latency, errors

Non-goals: full API management (billing, developer portal), multi-region replication.

2. Tech Stack

Layer	Choice	Reason
Language	Java 21, Spring Boot 3	Strong SDE signal, mature concurrency tooling
Shared state	Redis 7 + Lua scripts	Atomic ops, sub-millisecond latency
Redis client	Lettuce	Async, pooled, supports Cluster
Resilience	Resilience4j	Circuit breaker, timeouts
Load balancer	Nginx	Simulates multi-instance setup
Metrics	Micrometer, Prometheus, Grafana	Standard observability stack
Load testing	k6	p50/p95/p99 reporting, scriptable
Testing	JUnit 5, Mockito, Testcontainers	Real Redis in integration tests
Build/CI	Maven or Gradle, Docker, GitHub Actions	Reproducible builds
Deployment	Docker Compose on EC2/VPS (Kubernetes optional)	Live demo link

3. Architecture



Components

- **limiter-core:** RateLimiter interface, algorithm implementations, result types. No framework dependency.
- **limiter-redis:** Redis-backed implementations, Lua scripts, connection management.
- **gateway:** Spring Boot app with the rate limit filter, rule resolver, key extractor, reverse proxy to the backend.
- **demo-backend:** Simple upstream service to protect.
- **Rule store:** YAML at startup, Redis hash for runtime updates (stretch).

Request flow

1. Request arrives at a gateway instance via Nginx.
2. Key extractor builds the identity (API key header, JWT subject, or client IP).
3. Rule resolver matches route + tier to a rule (algorithm, limit, window).
4. Filter runs one Lua script call in Redis: check and consume atomically.
5. Allowed: forward upstream, attach rate limit headers. Denied: return 429 with Retry-After .
6. Record metrics (decision, latency, Redis errors).

Redis data model

Algorithm	Key	Structure	TTL
Fixed window	r1:fw:{rule}:{id}: {windowStart}	String counter	window length
Sliding log	r1:s1:{rule}:{id}	Sorted set (score = timestamp)	window length
Sliding counter	r1:sc:{rule}:{id}:{window}	Two counters (current, previous)	2x window

For Redis Cluster, wrap the identity in hash tags (`r1:{id}...`) so all keys for one client land on one shard.

Rule configuration (example)

```
rules:
- id: api-default
  match: /api/**
  key: api-key
  algorithm: token-bucket
  capacity: 100
  refillPerSecond: 10
- id: login
  match: /auth/login
  key: ip
  algorithm: sliding-window-counter
  limit: 5
  windowSeconds: 60
failurePolicy: fail-open
```

Key design decisions

- **Atomicity via Lua:** a client-side GET then SET races across instances; a Lua script executes atomically inside Redis.
- **Time source:** use Redis `TIME` inside scripts to avoid clock skew between gateway instances.
- **Failure policy:** fail-open with a local in-memory fallback limiter by default; fail-closed for sensitive routes like login.
- **Memory control:** every key has a TTL; no unbounded growth.
- **Hot keys:** short-lived local cache of "blocked until" decisions to cut Redis load from abusive clients.

4. Algorithms

Algorithm	Mechanism	Pros	Cons
Fixed window	INCR per window, EXPIRE	Simplest, cheapest	Up to 2x burst at window edges
Sliding window log	Sorted set of timestamps, trim old, count	Exact	Memory grows with limit
Sliding window counter	$\text{prev} * (1 - \text{elapsed/window}) + \text{current}$	Accurate enough, tiny memory	Approximate
Token bucket	Refill by elapsed time, spend per request	Controlled bursts, industry standard	Slightly more state

Token bucket Lua script (core)

```
-- KEYS[1] bucket key; ARGV: capacity, refill_per_sec, cost
local t = redis.call('TIME')
```

```

local now = t[1] * 1000 + math.floor(t[2] / 1000)
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or tonumber(ARGV[1])
local ts = tonumber(b[2]) or now
tokens = math.min(ARGV[1], tokens + (now - ts) / 1000 * ARGV[2])
local allowed = tokens >= tonumber(ARGV[3])
if allowed then tokens = tokens - ARGV[3] end
redis.call('HMSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(ARGV[1] / ARGV[2] * 2000))
return { allowed and 1 or 0, math.floor(tokens) }

```

Sliding window log (outline): ZREMRANGEBYSCORE key 0 (now - window), ZCARD key; if below limit then ZADD key now member and PEXPIRE, all in one script.

5. Phased Implementation Plan

Phase 0: Foundation (Days 1-3)

Tasks

- Write a one-page design doc from sections 1 and 3 of this document
- Create repo with multi-module layout, Docker Compose with Redis, GitHub Actions skeleton
- Set code style, logging, and test conventions

Exit criteria: docker compose up starts Redis; CI runs an empty build green.

Phase 1: Core algorithms, single node (Week 1)

Tasks

- Define `RateLimiter.tryAcquire(key, cost)` returning `RateLimitResult(allowed, remaining, resetAtMs)`
- Implement all four algorithms in memory using `ConcurrentHashMap` and atomic updates
- Inject a `Clock` abstraction so tests control time
- Unit tests for edge cases (window boundaries, refill, burst) and multi-threaded stress tests on one key
- JMH micro-benchmarks per algorithm

Exit criteria: all algorithms pass correctness and concurrency tests; baseline benchmark numbers recorded.

Phase 2: Distributed with Redis (Week 2)

Tasks

- Port each algorithm to a Lua script; load with `SCRIPT LOAD`, call with `EVALSHA`, handle `NOSCRIPT`
- Configure Lettuce pooling, command timeouts, and key naming
- Testcontainers integration tests: N threads simulating separate instances against one key; assert allowed total is at most the limit
- Build a deliberately non-atomic version to demonstrate the race, for the README

Exit criteria: limit holds under concurrent multi-client tests; naive version visibly fails.

Phase 3: Gateway and rules engine (Week 3)

Tasks

- Servlet filter or Spring Cloud Gateway filter calling the limiter
- Key extractors: API key, JWT subject, client IP (honor X-Forwarded-For safely)
- Rule loader and matcher (Ant-style path patterns, tier lookup)
- Standard headers and 429 body format
- Demo backend and proxy wiring

Exit criteria: end-to-end curl demo shows allowed requests, then 429 s, with correct headers and different limits per route.

Phase 4: Resilience and scaling (Week 4)

Tasks

- Resilience4j circuit breaker and timeout around Redis calls
- Fail-open and fail-closed policies, plus local in-memory fallback limiter
- Redis Cluster (3 masters) with hash tags; verify behavior
- Hot-key mitigation with a local blocked-until cache
- Admin API for runtime rule updates with Redis pub/sub invalidation (stretch)

Exit criteria: killing Redis mid-load keeps the gateway serving per policy; recovery is automatic.

Phase 5: Observability and load testing (Week 5)

Tasks

- Micrometer metrics: `rl_allowed_total`, `rl_blocked_total`, `rl_decision_latency`, `rl_redis_errors_total`, circuit breaker state
- Grafana dashboard exported as JSON in the repo
- k6 scenarios: steady load, spike, many unique keys, single hot key, Redis failure
- Benchmark table comparing algorithms (throughput, p50/p95/p99, Redis memory per key)

Exit criteria: documented benchmark report with real numbers and dashboard screenshots.

Phase 6: Ship it (Week 6)

Tasks

- Dockerfiles, Compose with Nginx + 3 gateway replicas + Redis + Prometheus + Grafana
- GitHub Actions: build, test with Testcontainers, build and push image
- Deploy to EC2/VPS with a public demo endpoint
- Final README, architecture diagram, demo GIF, design trade-offs section

Exit criteria: a stranger can run the project with one command and read results without asking you anything.

6. Testing Strategy

Level	What	Tools
Unit	Algorithm correctness with fake clock	JUnit 5
Concurrency	Many threads, one key, assert bound	ExecutorService, CountDownLatch

Integration	Real Redis, multi-instance simulation	Testcontainers
Contract	Headers, status codes, rule matching	MockMvc / WebTestClient
Chaos	Redis killed, latency injected	Docker stop, tc or Toxiproxy
Load	Throughput and tail latency	k6

7. Observability Plan

- **Metrics:** decisions by rule and outcome, decision latency histogram, Redis error rate, fallback activations
- **Logging:** structured JSON; log blocked requests at a sampled rate to avoid log floods
- **Dashboard panels:** req/s, block rate, p99 latency, Redis errors, circuit state per instance
- **Alerts (optional):** block rate spike, Redis error rate above threshold

8. Repository Structure

```

rate-limiter/
├── limiter-core/      # interfaces, algorithms, fake clock
├── limiter-redis/     # Lua scripts, Redis implementations
├── gateway/          # Spring Boot app, filter, rules
├── demo-backend/      # upstream service
├── loadtest/          # k6 scripts
├── deploy/            # compose, nginx.conf, grafana, prometheus
├── docs/              # design doc, diagrams, benchmarks
└── .github/workflows/

```

9. README Outline

1. Problem and goals, with a one-line result summary
2. Architecture diagram and request flow
3. Algorithm comparison table and when to use each
4. Atomicity demo (naive vs Lua)
5. Failure handling behavior
6. Benchmarks and dashboard screenshots
7. Quick start (docker compose up) and sample curl commands
8. Design decisions, trade-offs, and known limitations

10. Stretch Goals (pick one or two)

- Adaptive limits driven by backend latency or error rate
- gRPC interface so non-Java services can call the limiter
- Client-side consistent hashing instead of Redis Cluster
- Kubernetes deployment with HPA
- Distributed tracing with OpenTelemetry

11. Risks and Mitigations

Risk	Mitigation
Scope creep	Finish Phases 1-3 fully before any stretch goal
Skipped benchmarks	Protect Phase 5 time; numbers are the differentiator
Weak documentation	Treat README and diagrams as deliverables
Lua script bugs	Test each script in isolation with Testcontainers
Time pressure from placements	Phases 1-3 alone already give a presentable project

12. Interview Preparation

Be ready to explain: why Lua instead of GET then SET; token bucket versus sliding window trade-offs; behavior during a Redis outage or partition; scaling to millions of keys; the CAP implications of fail-open; how you measured performance and what you would optimize next.

13. Resume Bullets (replace X, Y with real numbers)

- Built a distributed rate limiter in Java/Spring Boot and Redis (Lua) with token bucket and sliding-window algorithms, enforcing limits atomically across multiple gateway instances.
- Sustained X req/s at p99 Y ms decision overhead in k6 load tests, with zero limit violations under concurrent multi-instance traffic.
- Added circuit breaker with fail-open fallback, Prometheus/Grafana monitoring, and Docker-based CI/CD; deployed live at [link].

14. Progress Checklist

- ☒ Phase 0: repo, CI, Compose, design doc
- ☒ Phase 1: in-memory algorithms and tests
- ☒ Phase 2: Redis Lua implementations and concurrency proof
- ☒ Phase 3: gateway, rules, headers, demo
- ☒ Phase 4: resilience, cluster, hot keys
- ☒ Phase 5: metrics, dashboards, load tests, benchmarks
- ☒ Phase 6: deployment, README, demo, resume bullets